

Day 15 – AWS Auto Scaling Group (ASG) with Nginx Deployment

Real-Time Scenario

A company hosts a website using Nginx on AWS.

Challenge

Normal Traffic → 2 Servers

High Traffic → More Servers Needed

Server Failure → Replace Failed Server

Low Traffic → Remove Extra Servers

Solution

AWS Auto Scaling Group (ASG)

Architecture

Users

|

v

Application Load Balancer

|

|

EC2-1

Nginx

|

v

Auto Scaling Group

Prerequisites

1. Create EC2 Instance

Launch:

Amazon Linux / Ubuntu

t2.micro

Install Nginx:

```
sudo apt update -y  
sudo apt install nginx -y
```

Verify:

```
systemctl status nginx
```

2. Create Website

```
sudo vi /var/www/html/index.html
```

Example:

```
<h1>Welcome to Auto Scaling Demo</h1>
```

3. Create AMI

Navigate:

EC2

- Instances
- Actions
- Image and Templates
- Create Image

Example:

AMI Name = nginx-ami

Purpose:

AMI = OS + Nginx + Website

Step 1 – Create Launch Template

Navigate:

EC2

- Launch Templates
- Create Launch Template

Provide:

Launch Template Name = ASG

AMI = nginx-ami

Instance Type = t2.micro

Security Group = HTTP + SSH

Key Pair = my-key

Click:

Create Launch Template

Step 2 – Create Auto Scaling Group

Navigate:

EC2

→ Auto Scaling Groups

→ Create Auto Scaling Group

Step 3 – Select Launch Template

Choose:

Launch Template = ASG

Version = Latest

Click:

Next

Step 4 – Network Configuration

Select:

Default VPC

Choose at least two subnets:

us-east-1a

us-east-1b

Purpose:

High Availability

Click:

Next

Step 5 – Attach Load Balancer

Select:

Attach to New Load Balancer

Choose:

Application Load Balancer

Target Group:

Create New Target Group

Port 80

Purpose:

Distribute traffic across instances

Click:

Next

Step 6 – Health Checks

Enable:

EC2 Health Checks

Enable:

Elastic Load Balancer Health Checks

Health Check Grace Period:

120 Seconds

Purpose:

Allow Nginx to start before health checks

Step 7 – Configure Group Size

Desired Capacity:

2

Minimum Capacity:

1

Maximum Capacity:

4

Meaning:

Normal = 2 Servers

Minimum = 1 Server

Maximum = 4 Servers

Step 8 – Configure Scaling Policy

Select:

Target Tracking Scaling Policy

Metric:

Average CPU Utilization

Target Value:

70%

Meaning:

CPU > 70%

Add Instance

CPU < 70%

Remove Instance

Step 9 – Instance Warmup

Enable:

Default Instance Warmup

Set:

120 Seconds

Purpose:

New server needs time to start Nginx

Step 10 – Review and Create

Review:

Launch Template

VPC

Subnets

Load Balancer

Health Checks

Scaling Policy

Click:

Create Auto Scaling Group

Validation

Navigate:

EC2

→ Auto Scaling Groups

→ Activity

Verify:

Launching EC2 Instance

Successful

Check EC2 Console:

2 Instances Running

Auto Scaling Flow

Normal Traffic

CPU = 20%

Instances = 2

High Traffic

CPU = 80%

CloudWatch detects:

CPU > 70%

ASG launches:

Instance-3

Low Traffic

CPU = 15%

ASG removes:

Instance-3

Server Failure

Instance-2 Failed

ASG detects:

Desired = 2

Running = 1

ASG launches:

New Instance

Result:

Running = 2

Interview Questions

What is Auto Scaling Group?

A service that automatically adds, removes, and replaces EC2 instances based on demand and health.

What is a Launch Template?

A blueprint containing:

AMI

Instance Type

Security Group

Key Pair

Why use a Load Balancer?

To distribute traffic across multiple EC2 instances.

Difference Between ALB and ASG?

ALB

ASG

Distributes Traffic Adds/Removes Instances

Handles Requests Handles Capacity

Why use Health Checks?

To automatically replace unhealthy instances.

Day 15 Summary

EC2 Instance



Install Nginx



Create AMI



Create Launch Template



Create Auto Scaling Group



Configure ALB



Configure Health Checks



Set Min/Desired/Max Capacity



Configure CPU-Based Scaling



Create ASG



Automatic Scale Out / Scale In